



Guía integral de estudio

Trabajo Práctico Integrador — Programación Web Avanzada

React + Vite · Node.js + Express · MariaDB

Tomas Oliva — E-commerce / Tienda Online

Cómo utilizar esta guía

No intentes memorizar cada línea. La defensa consiste en comprender responsabilidades y recorridos. Estudié primero los diagramas, luego practiqué tres acciones: cargar productos, ver un detalle y agregar al carrito.

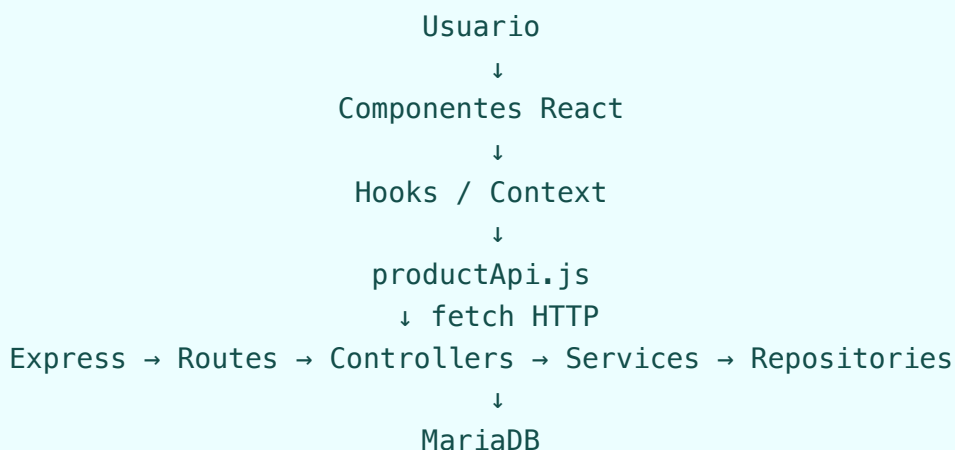
Idea principal: los datos bajan desde MariaDB hasta los componentes; las acciones del usuario suben desde los componentes hacia el estado.

Índice

1. Visión general y arquitectura
2. Frontend: entrada, componentes, hooks y API
3. Carrito, Context, localStorage y toast
4. CSS y posicionamiento
5. Backend en capas
6. HTTP, parámetros, paginación y errores
7. MariaDB, SQL e importación del CSV
8. Seguridad, limitaciones y mejoras
9. Simulación de examen con respuestas

1. Visión general

El proyecto es un e-commerce con una arquitectura cliente-servidor. React se ejecuta en el navegador, Express en Node.js y MariaDB almacena los datos.



Responsabilidades

Parte	Responsabilidad
Frontend	Mostrar la interfaz, mantener estados y detectar acciones del usuario.
Backend	Recibir HTTP, validar entradas, aplicar reglas y proteger el acceso a datos.
MariaDB	Almacenar productos y categorías de manera relacionada y persistente.

Presentación corta para el examen

“Es un e-commerce desarrollado con React y Vite en el frontend, Node.js y Express en el backend, y MariaDB como base. El frontend consume una API REST mediante fetch. El backend está separado en routes, controllers, services y repositories. Permite listar, filtrar, ordenar y paginar productos, consultar detalles y administrar un carrito persistido en localStorage mediante Context.”

2. El frontend

Entrada de la aplicación

index.html contiene `<div id="root">`. main.jsx crea la raíz de React y renderiza:

```
createRoot(document.getElementById('root')).render(  
  <CartProvider>  
    <App />  
  </CartProvider>,  
)
```

CartProvider envuelve a App, por lo que todos sus descendientes pueden consumir el carrito.

App como coordinador

App.jsx une la lógica y la presentación. Obtiene datos de los hooks y los entrega a componentes mediante props.

```
const cart = useCart()  
const { products, categories, page, selectProduct } = useProducts()  
  
<ProductList  
  products={products}  
  categories={categories}  
  onSelectProduct={selectProduct}  
>
```

Regla: hooks = lógica y estado; componentes = interfaz; App = conexión entre ambos.

Estado con useState

```
const [page, setPage] = useState(1)
```

page es el valor, setPage lo modifica y 1 es el valor inicial. Al cambiar el estado, React vuelve a renderizar.

Efectos con useEffect

```
useEffect(() => {  
  loadProducts()  
}, [categoryId, page, sortValue])
```

El efecto se ejecuta al montar y cuando cambia cualquiera de sus dependencias. Se usa para sincronizar el estado de React con una fuente externa, en este caso la API.

Dependencias	Comportamiento
Sin arreglo	Después de cada render.
[]	Una vez al montar.
[page]	Al montar y cuando cambia page.

useProducts

Concentra productos, categorías, página, filtro, orden, selección, loading y errores. Consulta la API y devuelve datos y acciones a App.

productApi → useProducts → App → ProductList → ProductCard

productApi y request

```

async function request(endpoint) {
  const response = await fetch(`${API_URL}${endpoint}`)
  return response.json()
}

export function getProductById(id) {
  return request(`/productos/${id}`)
}

```

getProductById(5) llama a request('/productos/5'). Request construye http://localhost:3000/productos/5. La conexión real ocurre en fetch.

Componentes y props

ProductList recibe un arreglo y genera una tarjeta por producto:

```

{products.map((product) => (
  <ProductCard key={product.id} product={product} />
))}

```

Las props bajan información. Las funciones recibidas por props permiten que una acción del hijo modifique un estado superior.

DATOS BAJAN

useProducts → App → ProductList → ProductCard

EVENTOS SUBEN

ProductCard → función recibida → estado superior

Recorrido: ver detalle

1. ProductCard ejecuta `onSelectProduct(product.id)`.
2. `selectProduct` actualiza `selectedProductId`.
3. Un `useEffect` detecta el ID.
4. `getProductById` hace fetch al backend.
5. La respuesta se guarda en `selectedProduct`.
6. App muestra ProductDetail.

3. Carrito y Context

Un elemento del carrito contiene el producto y la cantidad:

```
{
  product: { id: 1, name: 'Remera', price: 20 },
  quantity: 2
}
```

Agregar

```
      ¿El producto ya existe?
      └─ Sí → map aumenta quantity
      └─ No → se agrega con quantity 1
```

find busca el ID. map crea un arreglo nuevo modificando solo el elemento coincidente. Si no existe, spread conserva lo anterior y agrega una línea nueva.

Eliminar

```
currentItems.filter(
  item => item.product.id !== productId
)
```

filter conserva los elementos cuya condición da true. El producto con el ID eliminado produce false y desaparece del arreglo nuevo.

Actualizar cantidad

Si la cantidad es mayor que cero, map la reemplaza. Si es cero o negativa, se llama a removeProduct.

Context

```
      CartProvider crea y comparte el carrito
                ↓
      useCart lee el Context
                ↓
      App, Cart y productos usan sus funciones
```

const { addProduct } = useCart() desestructura solamente esa función del objeto compartido.

Persistencia

```
localStorage.setItem(KEY, JSON.stringify(items))
const items = JSON.parse(localStorage.getItem(KEY))
```

localStorage guarda texto. No es seguro ni confiable para una compra real: el usuario puede modificarlo.

Toast

CartProvider guarda una notificación al agregar o eliminar. Un efecto la cierra después de tres segundos y limpia el temporizador anterior para evitar comportamientos fuera de tiempo.

4. CSS y posicionamiento

JSX define estructura y orden; CSS define apariencia y distribución. `className="product-card"` se conecta con `.product-card`.

Clave: el padre posiciona a sus hijos. Primero busca el padre inmediato y comprueba si utiliza flujo normal, Flexbox o Grid.

Herramienta	Uso
Flujo normal	Los elementos de bloque aparecen uno debajo del otro.
Flexbox	Distribución principalmente en una fila o columna.
Grid	Distribución en filas y columnas.
Media query	Cambia reglas según el ancho de pantalla.

```
.products-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));  
  gap: 20px;  
}
```

Grid crea tantas columnas como entren, cada una de al menos 240px. En pantallas angostas las tarjetas bajan automáticamente.

5. Backend en capas

Route → Controller → Service → Repository → MariaDB

Capa	Pregunta que responde
Route	¿Qué función atiende esta URL?
Controller	¿Qué recibí y qué respuesta HTTP envío?
Service	¿Qué reglas y validaciones aplico?
Repository	¿Qué consulta SQL ejecuto?

Express

```
app.use(cors())
app.use(express.json())
app.use('/productos', productRoutes)
app.use('/categorias', categoryRoutes)
```

CORS permite al frontend de otro puerto comunicarse. `express.json` prepara la lectura de cuerpos JSON.

Parámetros HTTP

Tipo	Ejemplo	Uso
params	/productos/5	Identificar un recurso.
query	?limit=12&offset=24	Filtrar, ordenar o paginar una colección.
body	JSON dentro de POST/PUT	Enviar datos para crear o modificar.

Paginación

```
offset = (page - 1) * limit
```

Página	LIMIT	OFFSET	Interpretación
1	12	0	No salta filas.
2	12	12	Salta las primeras 12.
3	12	24	Salta las primeras 24.

El total se obtiene con `COUNT` y las páginas se calculan con `Math.ceil(total / limit)`.

mysql2/promise

Es la interfaz de mysql2 basada en promesas. Permite ejecutar consultas con `await pool.query(...)`. El pool reutiliza conexiones y atiende consultas concurrentes.

Manejo de errores

El service genera errores esperados, el controller usa `next(error)` y el middleware central responde de forma uniforme.

- 400: petición inválida.
- 404: recurso válido pero inexistente.
- 500: error interno inesperado.

6. MariaDB y SQL

```
categorias.id ← productos.categoria_id
```

La clave foránea evita categorías inexistentes y reduce duplicación de nombres.

```
SELECT
  p.id,
  p.nombre AS name,
  p.precio AS price,
  c.nombre AS category
FROM productos p
INNER JOIN categorias c
  ON p.categoria_id = c.id
WHERE p.categoria_id = ?
ORDER BY p.precio ASC, p.id ASC
LIMIT ? OFFSET ?;
```

Los signos `?` son placeholders. Los valores se envían separados para reducir el riesgo de inyección SQL. El ID secundario estabiliza el orden cuando hay precios iguales.

Importación del CSV

```
CSV → productos_raw → limpieza → categorias + productos
```

Es un proceso ETL: extraer, transformar y cargar. `DISTINCT` crea categorías únicas, `TRIM` limpia espacios y `JOIN` obtiene el ID relacionado.

7. Seguridad, limitaciones y mejoras

- El usuario de MariaDB tiene solo `SELECT`: principio de mínimo privilegio.
- `.env` evita subir credenciales al repositorio.
- Las consultas parametrizadas protegen valores.

- El ordenamiento usa una lista blanca porque los nombres de columna no admiten placeholders.
- CORS abierto es útil en desarrollo, pero debería restringirse en producción.
- localStorage puede ser manipulado; el backend debe recalcular una compra real.
- OFFSET funciona con 1000 productos, pero puede ser costoso con millones.
- No hay usuarios, autenticación, pedidos ni operaciones de escritura.
- Para ser una PWA completa faltan manifest, service worker y estrategia offline.

8. Simulación de examen

1. Explicá qué sucede al cambiar de página.

ProductList ejecuta una función recibida por props. Esa función cambia page en useProducts. Como page es dependencia del efecto, se calcula un nuevo offset y getProducts ejecuta fetch. Express recibe req.query, pasa por route, controller, service y repository. MariaDB aplica LIMIT/OFFSET. La respuesta vuelve como JSON, se guarda con setProducts y React vuelve a renderizar.

2. ¿Por qué no usar req.query.sortBy directamente en SQL?

Los nombres de columnas no se parametrizan con ?. Insertar texto del usuario sería riesgoso. El proyecto utiliza una lista blanca que mapea name y price a columnas conocidas.

3. ¿Qué pasa si dos peticiones de filtros llegan fuera de orden?

Existe una posible condición de carrera: una respuesta vieja podría sobrescribir una nueva. Se mejoraría con AbortController en la limpieza del useEffect.

4. ¿Por qué setItems recibe una función?

Porque el estado nuevo depende del anterior. La actualización funcional recibe la versión más reciente aunque React agrupe actualizaciones, evitando perder incrementos rápidos.

5. ¿Por qué no modificar items con push?

push muta el arreglo existente. React trabaja mejor con referencias nuevas; map, filter y spread respetan inmutabilidad y hacen predecibles los renders.

6. ¿Cuál es la diferencia entre un error de red y un 404?

Si fetch no puede conectarse, rechaza la promesa y entra al catch. Un 404 sí es una respuesta HTTP: fetch se resuelve, pero response.ok es false.

7. ¿Promise.all garantiza consistencia?

Ejecuta listado y conteo en paralelo, pero con escrituras concurrentes podrían observar estados distintos. Aquí la API es de solo lectura; para consistencia estricta se consideraría una transacción.

8. ¿Por qué ordenar también por ID?

Desempata productos con igual nombre o precio. Un orden estable evita repeticiones o saltos entre páginas.

9. ¿Qué limitación tiene Number para precios?

JavaScript usa punto flotante y puede producir imprecisiones. En un sistema financiero convendría usar centavos enteros o una librería decimal.

10. ¿Por qué Context puede producir renders adicionales?

El objeto value cambia de identidad en cada render, por lo que los consumidores se actualizan. En esta escala no es problemático; se podría optimizar con useMemo, useCallback o contextos separados.

11. ¿Qué diferencia hay entre 400 y 404 en /productos/:id?

“hola” o -2 son IDs mal formados y producen 400. Un entero positivo que no existe es una solicitud válida sobre un recurso ausente y produce 404.

12. ¿Por qué el repository no devuelve res.status(404)?

El repository no debe conocer HTTP; solamente devuelve datos o null. El controller interpreta el resultado y decide la respuesta HTTP.

13. ¿Qué garantiza la clave foránea?

Que productos.categoria_id apunte a una categoría existente, manteniendo integridad referencial.

14. ¿El proyecto es una PWA completa?

Todavía no. Está hecho con React/Vite, pero para una PWA instalable faltan manifest, service worker, caché y estrategia offline.

15. ¿Cuál debe ser la fuente de verdad del precio?

MariaDB a través del backend. localStorage es manipulable y solo sirve para experiencia visual; un checkout real debe recuperar y recalcular precios en el servidor.

9. Respuesta integradora final

“El usuario interactúa con componentes React. Las acciones ejecutan funciones recibidas por props o Context y modifican estados. Cuando un estado usado como dependencia cambia, useEffect sincroniza el frontend con la API. productApi construye la URL y fetch envía HTTP. Express dirige la petición desde routes hasta controllers, services y repositories. El repository ejecuta SQL parametrizado en MariaDB. La respuesta vuelve como JSON, React actualiza sus estados y renderiza la interfaz. El carrito es diferente: vive en Context y se persiste localmente en el navegador.”

10. Repaso de un minuto

- **Frontend:** estado, efectos, props, Context y componentes.
- **Conexión:** fetch a una API REST.
- **Backend:** route → controller → service → repository.
- **Base:** productos relacionados con categorías.
- **Paginación:** limit determina cantidad; offset salta filas.
- **Carrito:** find/map para agregar, filter para eliminar.
- **Seguridad:** .env, usuario SELECT, placeholders y listas blancas.

Documento de estudio generado para el TP Integrador PWA de Tomas Oliva.